

Week 2 Lab: Modelling

Wakana

part A

original

```
import numpy as np
import matplotlib.pyplot as plt
def decay_numerical(C0, k, dt, t_end):
    """Return arrays of time and concentration for forward-Euler tracer decay."""
    n = int(t_end / dt)
    C = C0
    ts = [0.0]
    Cs = [C0]
    for i in range(n):
        C = C + (-k * C) * dt          # forward difference
        ts.append((i + 1) * dt)
        Cs.append(C)
    return np.array(ts), np.array(Cs)

# The exact answer, for comparison
C0, k, t_end = 100.0, 1.0, 6.0
t_fine = np.linspace(0, t_end, 200)
exact = C0 * np.exp(-k * t_fine)

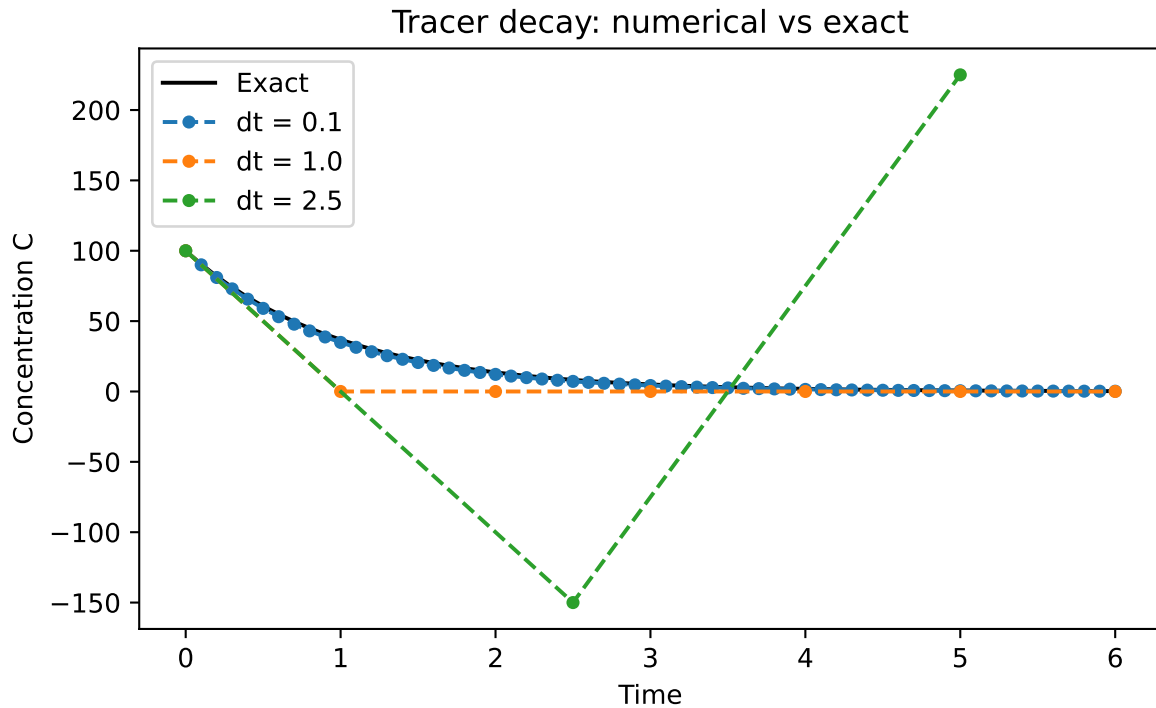
fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(t_fine, exact, "k-", label="Exact")

for dt in [0.1, 1.0, 2.5]:
    ts, Cs = decay_numerical(C0, k, dt, t_end)
    ax.plot(ts, Cs, "o--", markersize=4, label=f"dt = {dt}")
```

```

ax.set_xlabel("Time")
ax.set_ylabel("Concentration C")
ax.set_title("Tracer decay: numerical vs exact")
ax.legend()
plt.show()

```



warmup

```

def decay_numerical(C0, k, dt, t_end):
    """Return arrays of time and concentration for forward-Euler tracer decay."""
    n = int(t_end / dt)
    C = C0
    ts = [0.0]
    Cs = [C0]
    for i in range(n):
        C = C + (-k * C) * dt          # forward difference
        ts.append((i + 1) * dt)
        Cs.append(C)

```

```

    return np.array(ts), np.array(Cs)

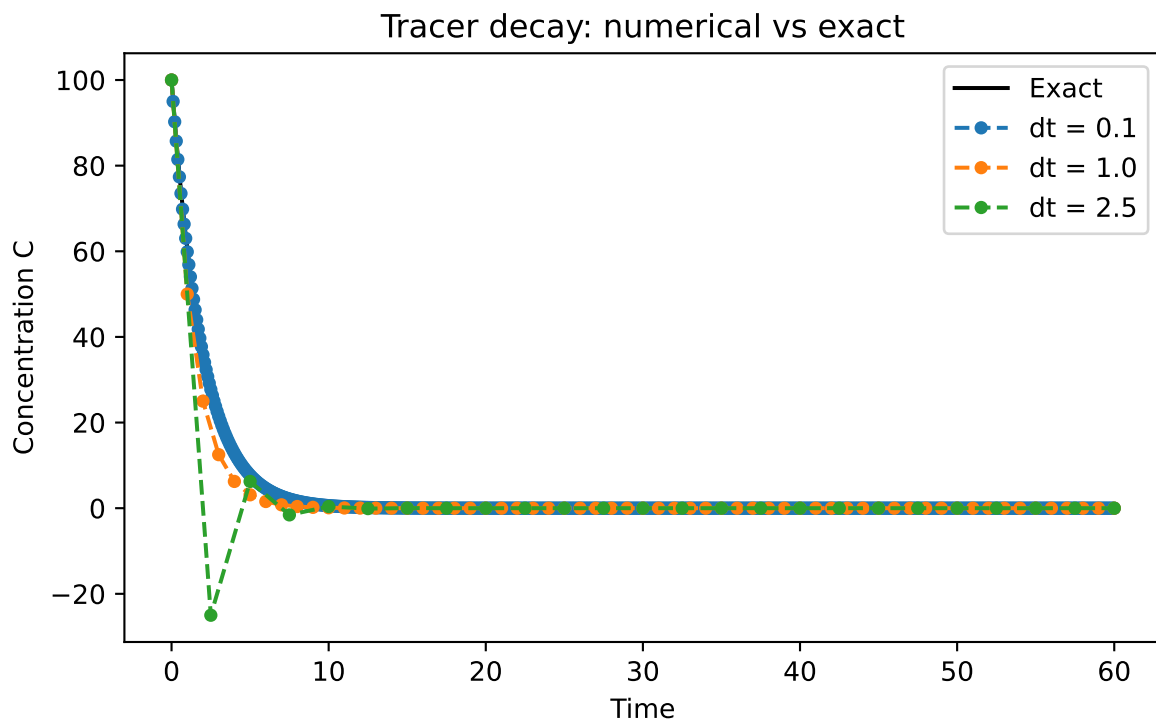
# The exact answer, for comparison
C0, k, t_end = 100.0, 0.5, 60
dt = 1.0
t_fine = np.linspace(0, t_end, 200)
exact = C0 * np.exp(-k * t_fine)

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(t_fine, exact, "k-", label="Exact")

for dt in [0.1, 1.0, 2.5]:
    ts, Cs = decay_numerical(C0, k, dt, t_end)
    ax.plot(ts, Cs, "o--", markersize=4, label=f"dt = {dt}")

ax.set_xlabel("Time")
ax.set_ylabel("Concentration C")
ax.set_title("Tracer decay: numerical vs exact")
ax.legend()
plt.show()

```



task 1.1

```
def decay_numerical(C0, k, dt, t_end):
    """Return arrays of time and concentration for forward-Euler tracer decay."""
    n = int(t_end / dt)
    C = C0
    ts = [0.0]
    Cs = [C0]
    for i in range(n):
        C = C + (-k * C) * dt          # forward difference
        ts.append((i + 1) * dt)
        Cs.append(C)
    return np.array(ts), np.array(Cs)

# The exact answer, for comparison
C0, k, t_end = 100.0, 0.1, 60
dt = 1.0
t_fine = np.linspace(0, t_end, 200)
exact = C0 * np.exp(-k * t_fine)

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(t_fine, exact, "k-", label="Exact")

for dt in [1.0]:
    ts, Cs = decay_numerical(C0, k, dt, t_end)
    ax.plot(ts, Cs, "o--", markersize=4, label=f"dt = {dt}")

ax.set_xlabel("Time")
ax.set_ylabel("Concentration C")
ax.set_title("Tracer decay: numerical vs exact")
ax.legend()
plt.show()

def decay_numerical(C0, k, dt, t_end):
    """Return arrays of time and concentration for forward-Euler tracer decay."""
    n = int(t_end / dt)
    C = C0
    ts = [0.0]
    Cs = [C0]
    for i in range(n):
```

```

        C = C + (-k * C) * dt          # forward difference
        ts.append((i + 1) * dt)
        Cs.append(C)
    return np.array(ts), np.array(Cs)

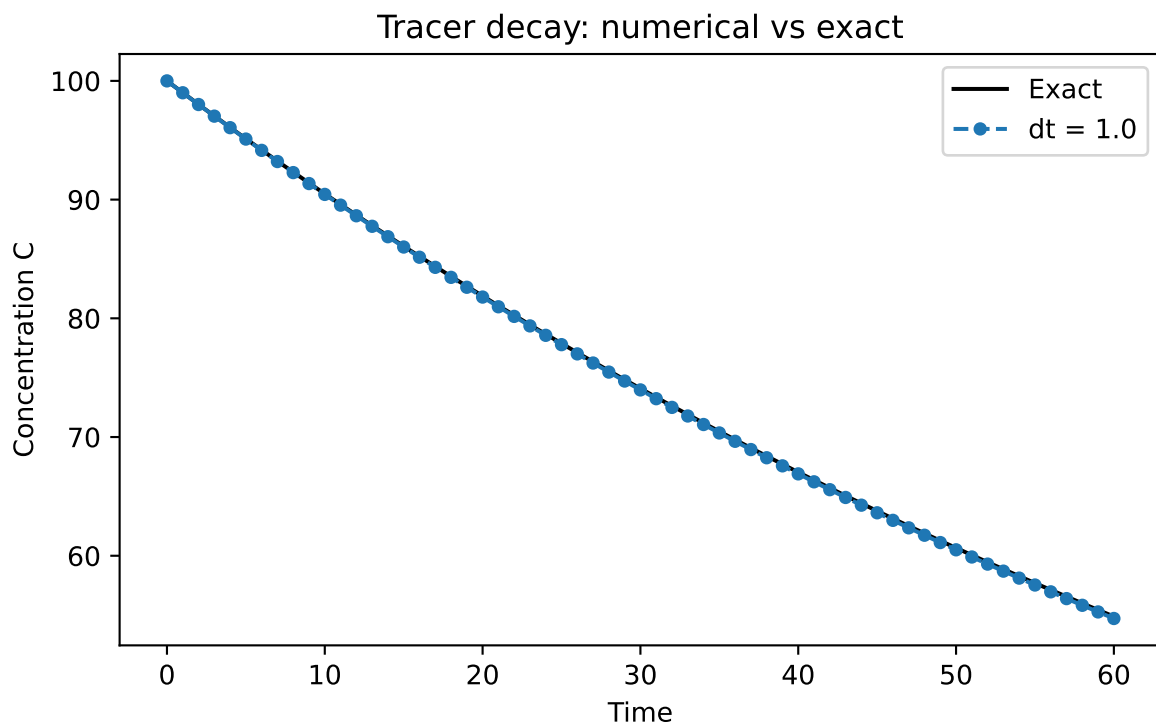
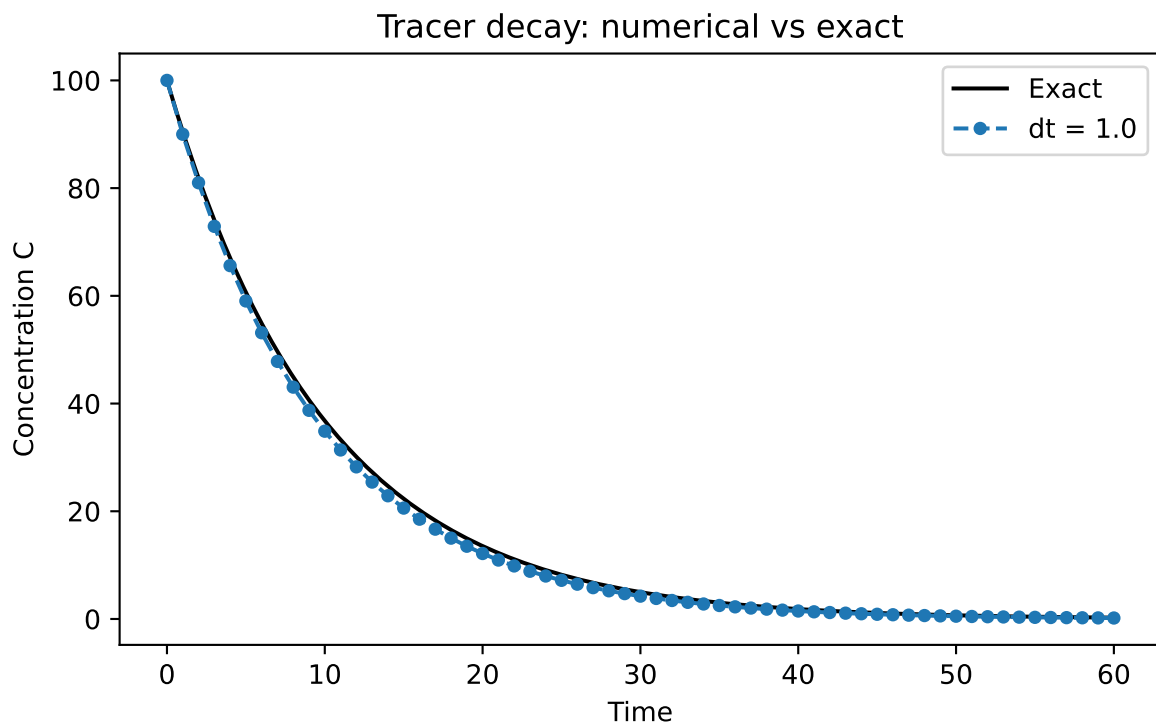
# The exact answer, for comparison
C0, k, t_end = 100.0, 0.01, 60
dt = 1.0
t_fine = np.linspace(0, t_end, 200)
exact = C0 * np.exp(-k * t_fine)

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(t_fine, exact, "k-", label="Exact")

for dt in [1.0]:
    ts, Cs = decay_numerical(C0, k, dt, t_end)
    ax.plot(ts, Cs, "o--", markersize=4, label=f"dt = {dt}")

ax.set_xlabel("Time")
ax.set_ylabel("Concentration C")
ax.set_title("Tracer decay: numerical vs exact")
ax.legend()
plt.show()

```



decay rate k describes the rate of decay of the concentration c , where the smaller the value is, the slower the decay is, and the larger the value c is the faster the decay rate is

task 1.2

```
def decay_numerical(C0, k, dt, t_end):
    """Return arrays of time and concentration for forward-Euler tracer decay."""
    n = int(t_end / dt)
    C = C0
    ts = [0.0]
    Cs = [C0]
    for i in range(n):
        C = C + (-k * C) * dt          # forward difference
        ts.append((i + 1) * dt)
        Cs.append(C)
    return np.array(ts), np.array(Cs)

# The exact answer, for comparison
C0, k, t_end = 100.0, 0.5, 60
dt = 2.0
t_fine = np.linspace(0, t_end, 200)
exact = C0 * np.exp(-k * t_fine)

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(t_fine, exact, "k-", label="Exact")

for dt in [2.0]:
    ts, Cs = decay_numerical(C0, k, dt, t_end)
    ax.plot(ts, Cs, "o--", markersize=4, label=f"dt = {dt}")

ax.set_xlabel("Time")
ax.set_ylabel("Concentration C")
ax.set_title("Tracer decay: numerical vs exact")
ax.legend()
plt.show()

def decay_numerical(C0, k, dt, t_end):
    """Return arrays of time and concentration for forward-Euler tracer decay."""
    n = int(t_end / dt)
    C = C0
```

```

ts = [0.0]
Cs = [C0]
for i in range(n):
    C = C + (-k * C) * dt          # forward difference
    ts.append((i + 1) * dt)
    Cs.append(C)
return np.array(ts), np.array(Cs)

# The exact answer, for comparison
C0, k, t_end = 100.0, 0.5, 60
dt = 5.0
t_fine = np.linspace(0, t_end, 200)
exact = C0 * np.exp(-k * t_fine)

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(t_fine, exact, "k-", label="Exact")

for dt in [5.0]:
    ts, Cs = decay_numerical(C0, k, dt, t_end)
    ax.plot(ts, Cs, "o--", markersize=4, label=f"dt = {dt}")

ax.set_xlabel("Time")
ax.set_ylabel("Concentration C")
ax.set_title("Tracer decay: numerical vs exact")
ax.legend()
plt.show()

def decay_numerical(C0, k, dt, t_end):
    """Return arrays of time and concentration for forward-Euler tracer decay."""
    n = int(t_end / dt)
    C = C0
    ts = [0.0]
    Cs = [C0]
    for i in range(n):
        C = C + (-k * C) * dt          # forward difference
        ts.append((i + 1) * dt)
        Cs.append(C)
    return np.array(ts), np.array(Cs)

# The exact answer, for comparison
C0, k, t_end = 100.0, 0.5, 60
dt = 1.5

```



```

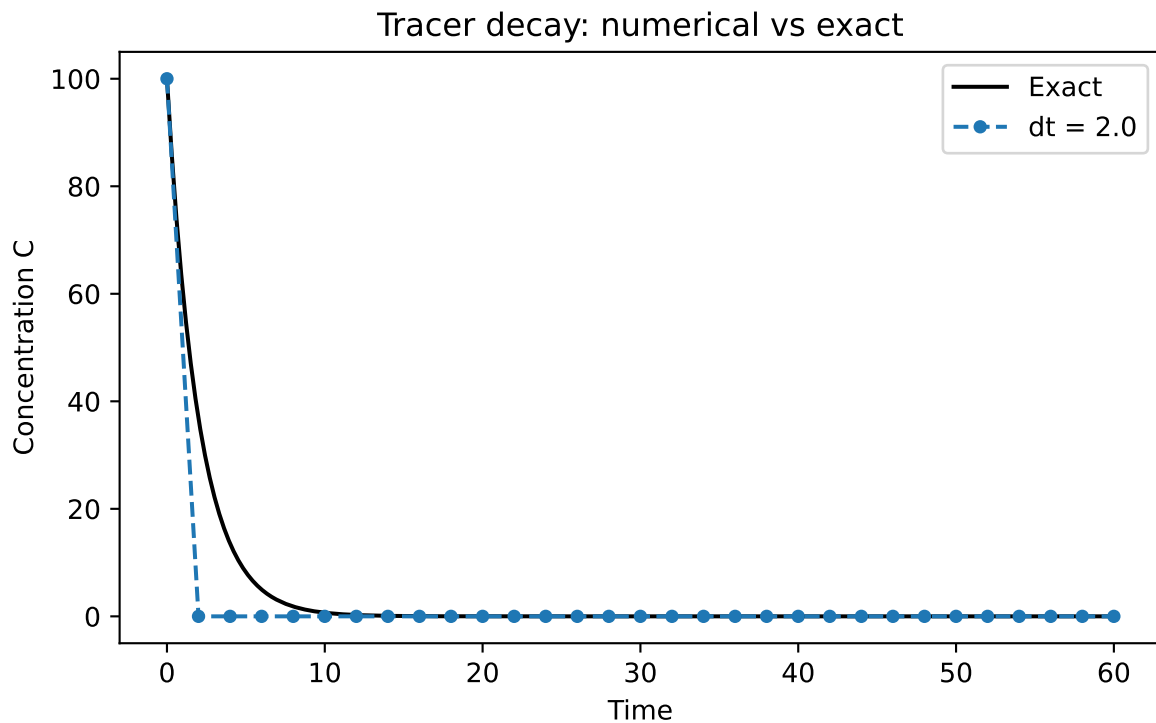
t_fine = np.linspace(0, t_end, 200)
exact = C0 * np.exp(-k * t_fine)

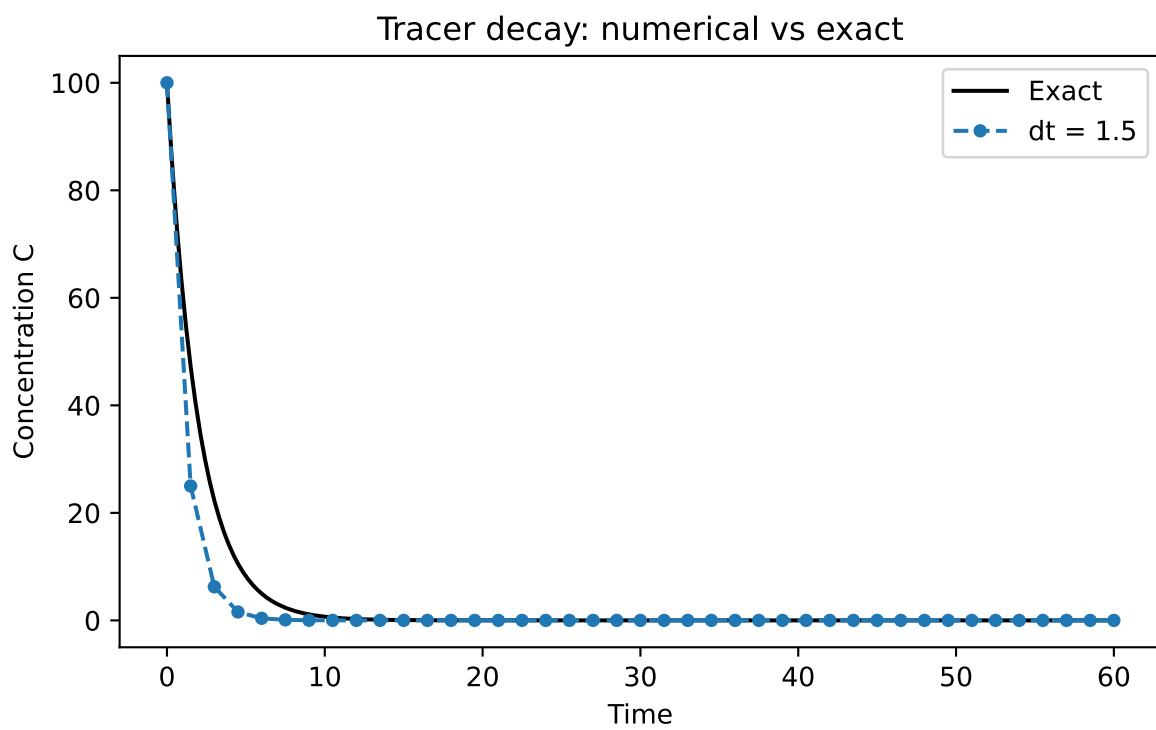
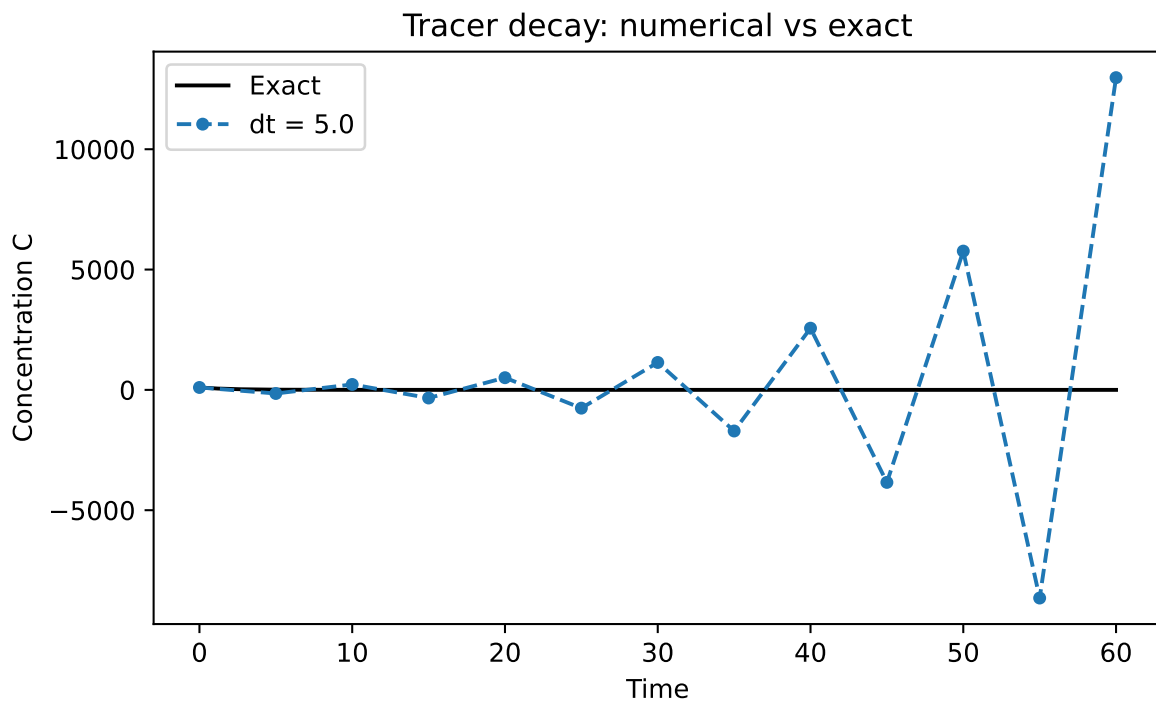
fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(t_fine, exact, "k-", label="Exact")

for dt in [1.5]:
    ts, Cs = decay_numerical(C0, k, dt, t_end)
    ax.plot(ts, Cs, "o--", markersize=4, label=f"dt = {dt}")

ax.set_xlabel("Time")
ax.set_ylabel("Concentration C")
ax.set_title("Tracer decay: numerical vs exact")
ax.legend()
plt.show()

```





the value of dt starts to disagree almost immediately after 1.0. at 2, although it looks accurate, it drops to 0 immediately and follows a steady 0, whereas for dt of 5, it fluctuates to the two extremes throughout the graph, exponentially getting more extreme. maybe around 1.5 when the line starts to disagree with the main line.

task 1.3

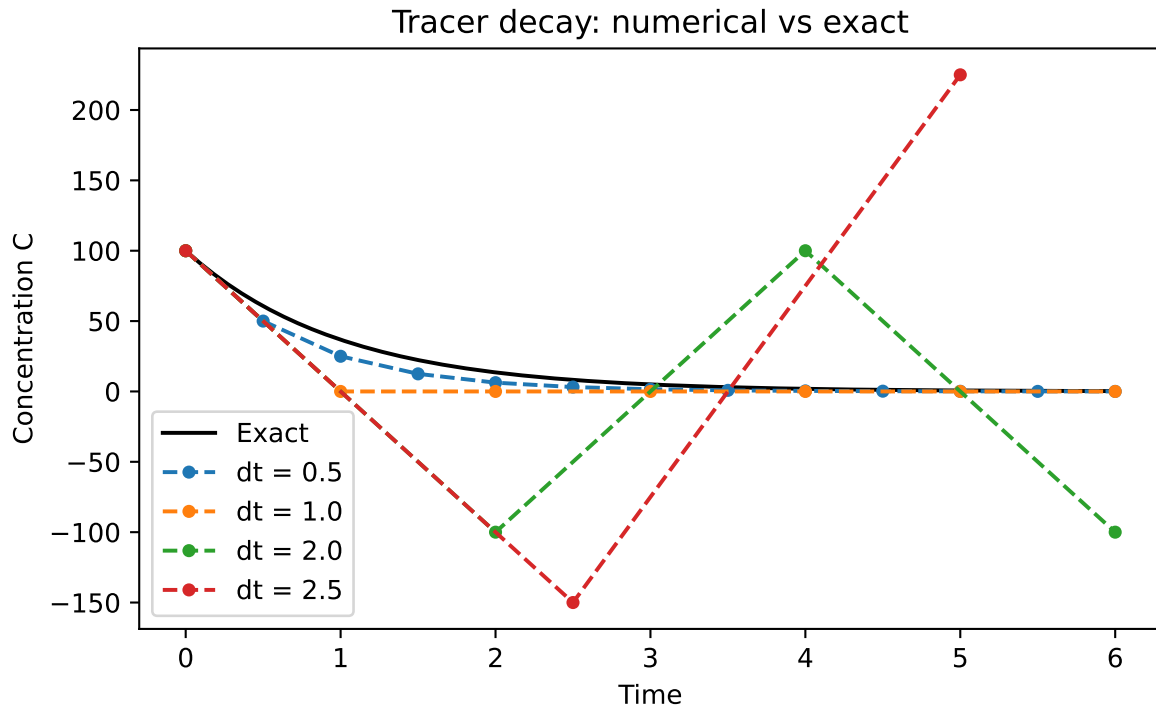
```
import numpy as np
import matplotlib.pyplot as plt
def decay_numerical(C0, k, dt, t_end):
    """Return arrays of time and concentration for forward-Euler tracer decay."""
    n = int(t_end / dt)
    C = C0
    ts = [0.0]
    Cs = [C0]
    for i in range(n):
        C = C + (-k * C) * dt          # forward difference
        ts.append((i + 1) * dt)
        Cs.append(C)
    return np.array(ts), np.array(Cs)

# The exact answer, for comparison
C0, k, t_end = 100.0, 1.0, 6.0
t_fine = np.linspace(0, t_end, 200)
exact = C0 * np.exp(-k * t_fine)

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(t_fine, exact, "k-", label="Exact")

for dt in [0.5, 1.0, 2.0, 2.5]:
    ts, Cs = decay_numerical(C0, k, dt, t_end)
    ax.plot(ts, Cs, "o--", markersize=4, label=f"dt = {dt}")

ax.set_xlabel("Time")
ax.set_ylabel("Concentration C")
ax.set_title("Tracer decay: numerical vs exact")
ax.legend()
plt.show()
```



$dt=1.0$ is the largest value which doesn't go below the negative line. The values above (2.0 and 2.5) both go below 0, and then immediately jump above, giving us an extreme pattern.

task 1.4

The value of dt determines how often the rate of decay is being considered within the equation, and when the value is larger than 1, it thinks that the removing percentage is larger than what exists within the system, therefore removes a larger amount than there is. That causes the values to go below 0, and because the value that it first produced is fed into the system again, it becomes a positive feedback loop.

part B

original

```
import numpy as np
import matplotlib.pyplot as plt
```

```

# ---- Parameters ----
V = 1.0e6          # lake volume (m^3)
Q = 5.0e3          # river inflow (m^3 per day)
C_in = 10.0        # incoming concentration (mg/L)
k = 0.05           # in-lake decay rate (per day)

# ---- Time settings ----
dt = 1.0           # time step (days)
days = 200

# ---- Initial condition ----
C = 0.0            # clean lake

# ---- Storage ----
history = [C]

# ---- Time loop ----
for _ in range(days):
    dCdt = (Q * C_in - Q * C - k * V * C) / V  # the rate of change
    C = C + dCdt * dt                          # forward difference
    history.append(C)

# ---- A quick look at the numbers ----
print(f"Started at {history[0]:.2f} mg/L")
print(f"After 10 days: {history[10]:.2f} mg/L")
print(f"After 50 days: {history[50]:.2f} mg/L")
print(f"After {days} days: {history[-1]:.2f} mg/L")

```

Started at 0.00 mg/L
 After 10 days: 0.39 mg/L
 After 50 days: 0.86 mg/L
 After 200 days: 0.91 mg/L

task 2.1

```

V = 1e6
Q = 5000
C_in = 10
k = 0.05

```

```

# ---- Time settings ----
dt = 1.0          # time step (days)
days = 200

# ---- Initial condition ----
C = 0.0          # clean lake

# ---- Storage ----
history = [C]

# ---- Time loop ----
for _ in range(days):
    dCdt = (Q * C_in - Q * C - k * V * C) / V  # the rate of change
    C = C + dCdt * dt                          # forward difference
    history.append(C)

# ---- A quick look at the numbers ----
print(f"Started at {history[0]:.2f} mg/L")
print(f"After 10 days: {history[10]:.2f} mg/L")
print(f"After 50 days: {history[50]:.2f} mg/L")
print(f"After {days} days: {history[-1]:.2f} mg/L")

```

```

Started at 0.00 mg/L
After 10 days: 0.39 mg/L
After 50 days: 0.86 mg/L
After 200 days: 0.91 mg/L

```

the value which the original put out was 0.9, whereas the task 2.1 produced 0.9, therefore they are the same.

task 2.2

```

import numpy as np
import matplotlib.pyplot as plt

# ---- Parameters ----
V = 1.0e6          # lake volume (m3)
Q = 5.0e3          # river inflow (m3 per day)
C_in = 10.0        # incoming concentration (mg/L)
k = 0.05           # in-lake decay rate (per day)

```

```

# ---- Time settings ----
dt = 1.0          # time step (days)
days = 200

# ---- Initial condition ----
C = 20.0

# ---- Storage ----
history = [C]

# ---- Time loop ----
for _ in range(days):
    dCdt = (Q * C_in - Q * C - k * V * C) / V  # the rate of change
    C = C + dCdt * dt                          # forward difference
    history.append(C)

# ---- A quick look at the numbers ----
print(f"Started at {history[0]:.2f} mg/L")
print(f"After 10 days: {history[10]:.2f} mg/L")
print(f"After 50 days: {history[50]:.2f} mg/L")
print(f"After {days} days: {history[-1]:.2f} mg/L")

```

```

Started at 20.00 mg/L
After 10 days: 11.75 mg/L
After 50 days: 2.04 mg/L
After 200 days: 0.91 mg/L

```

day 10 and day 50 becomes much higher than 1, which can be explained by the steady input of pollutants as well as the pollutants which were already within the water column. This was likely lowered as the pollutant decayed over time, reaching the same steady point

task 2.3

```

V = 1.0e6          # lake volume (m3)
Q = 5.0e3 * 2      # river inflow (m3 per day)
C_in = 10.0        # incoming concentration (mg/L)
k = 0.05           # in-lake decay rate (per day)

# ---- Time settings ----

```

```

dt = 1.0          # time step (days)
days = 200

# ---- Initial condition ----
C = 0.0          # clean lake

# ---- Storage ----
history = [C]

# ---- Time loop ----
for _ in range(days):
    dCdt = (Q * C_in - Q * C - k * V * C) / V # the rate of change
    C = C + dCdt * dt                         # forward difference
    history.append(C)

# ---- A quick look at the numbers ----
print(f"Started at {history[0]:.2f} mg/L")
print(f"After 10 days: {history[10]:.2f} mg/L")
print(f"After 50 days: {history[50]:.2f} mg/L")
print(f"After {days} days: {history[-1]:.2f} mg/L")

```

```

Started at 0.00 mg/L
After 10 days: 0.77 mg/L
After 50 days: 1.59 mg/L
After 200 days: 1.67 mg/L

```

Increasing Q increases C as the larger the inflow, the more pollutants come into the lake, while the decay rate remains the same, therefore the resulting pollutants is much higher

task 2.4

```

# ---- Parameters ----
V = 1.0e6        # lake volume (m^3)
Q = 5.0e3        # river inflow (m^3 per day)
C_in = 10.0      # incoming concentration (mg/L)
k = 0.05         # in-lake decay rate (per day)

# ---- Time settings ----
dt = 1.0         # time step (days)

```



```

days = 200

# ---- Initial condition ----
C = 0.0          # clean lake

# ---- Storage ----
history = [C]

# ---- Time loop ----
for _ in range(days):
    dCdt = (Q * C_in - Q * C - k * V * C) / V    # the rate of change
    C = C + dCdt * dt                            # forward difference
    history.append(C)

# ---- A quick look at the numbers ----
print(f"Started at {history[0]:.2f} mg/L")
print(f"After 10 days: {history[10]:.2f} mg/L")
print(f"After 50 days: {history[50]:.2f} mg/L")
print(f"After {days} days: {history[-1]:.2f} mg/L")

fig, ax = plt.subplots(figsize=(7, 4))

for k_test in [0.02, 0.05, 0.1, 0.2]:
    C = 0.0
    history = [C]
    for _ in range(days):
        dCdt = (Q * C_in - Q * C - k_test * V * C) / V
        C = C + dCdt * dt
        history.append(C)
    ax.plot(history, label=f"k = {k_test}")

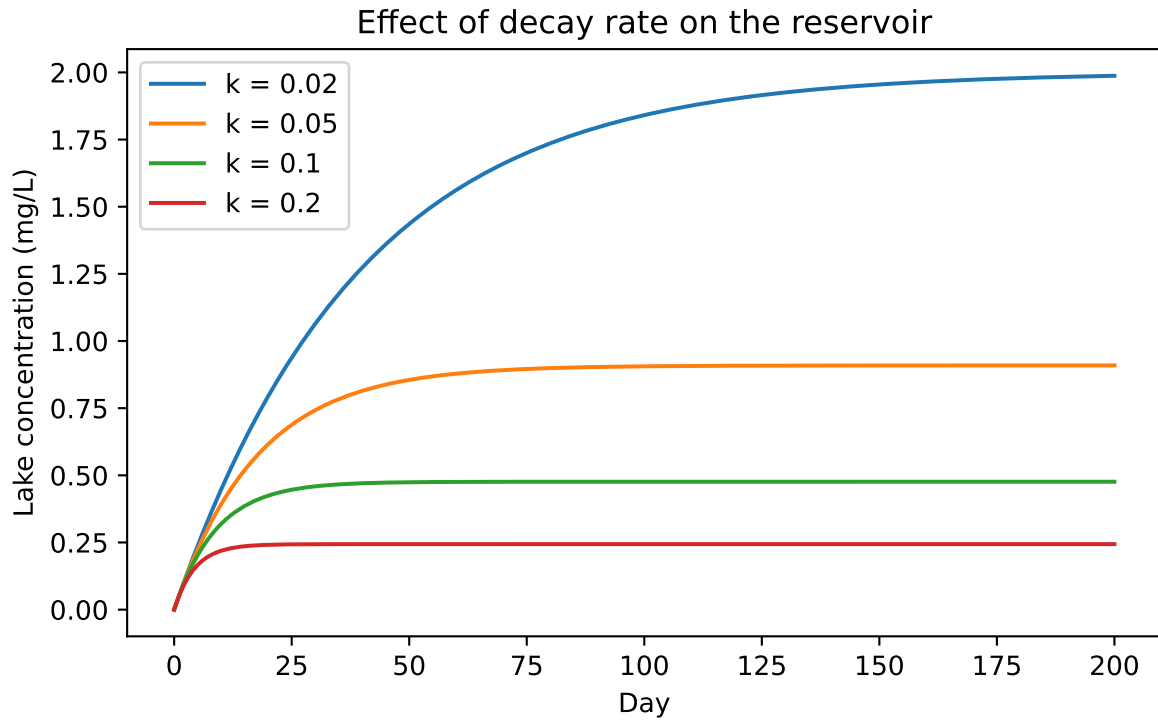
ax.set_xlabel("Day")
ax.set_ylabel("Lake concentration (mg/L)")
ax.set_title("Effect of decay rate on the reservoir")
ax.legend()
plt.show()

```

```

Started at 0.00 mg/L
After 10 days: 0.39 mg/L
After 50 days: 0.86 mg/L
After 200 days: 0.91 mg/L

```



reflection

I found the difference of language compared to R to be the trickiest. the difference in format really threw me off, and the terminal and how it looks also threw me off. If i had another hour, I would change it so that i dont have to change the value in each square, so that i dont have to copy paste the whole block and edit them each time. I would hve tried to make an automated system I think.